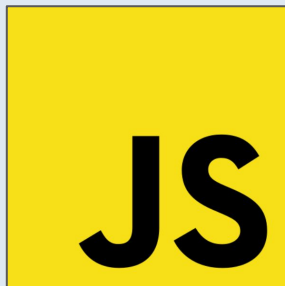


Mòdul 06



Institut TIC
de Barcelona



Desenvolupament web en entorn
client

RA4 - Estructures definides pel programador.
Objectes

1. OOP (Programació orientat a objectes)
 - a. Objecte vs classe
 - b. Objectes
 - c. Classes
 - i. Instàncies
 - ii. Getter/Setters
 - iii. Públic/Privat
 - iv. Herència
 - v. Static
 - d. Prototype
2. Array/Set
 - a. Mètodes array
3. Destructuració arrays i objectes
4. Spread Operator (...)

- Un objecte és una col·lecció de propietats i tipus.
- Una propietat és l'associació *key : value*
- Un *value* pot ser:
 - *Variables* de tipus primitives (*number, string, boolean, undefined, ...*)
 - *Objectes*: *objectes, arrays, dates,...*
 - *Funcions i expressions*

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  edat: 33,  
  teCarnetConduir: true,  
  altura: 1.79,  
};
```

```
const persona = {  
  nom: "Jhon",  
  cognom: "Smith",  
  teCarnetConduir: true,  
  altura: 1.79,  
  edat: 2024 - 1999,  
  amics: ["Marta", "Joan"],  
  direccio: {  
    ciutat: "Barcelona",  
    cp: "09005",  
  },  
  esMajorEdat: function () {  
    return this.edat > 18;  
  },  
};
```

variables primitives

expressions i operacions

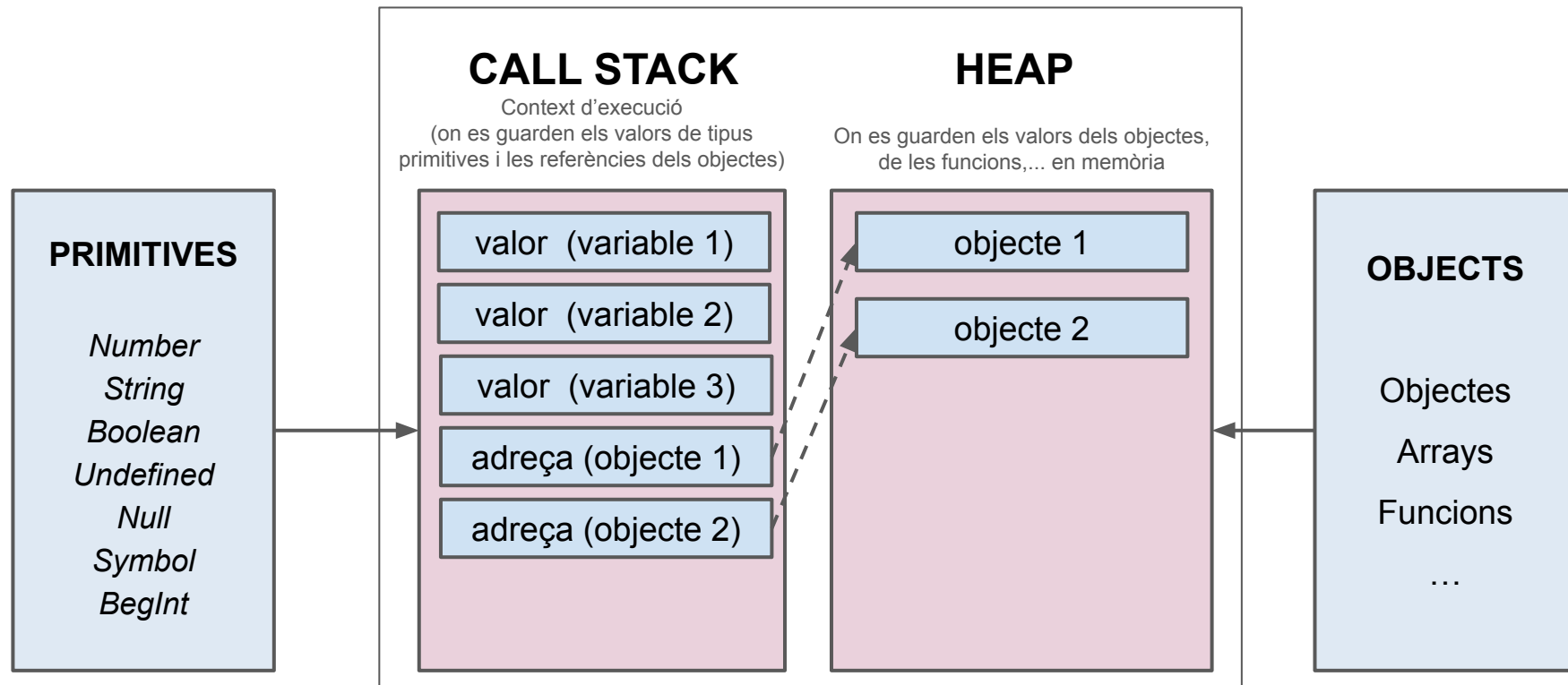
array

objecte

function expression

Com funciona javascript?

JS ENGINE



```
// VARIABLES PRIMITIVES
```

```
let edat = 35;
```

```
let edatNou = edat;
```

```
edat = 36;
```

```
// VARIABLES OBJECTES
```

```
const persona_template = {
```

```
  nom: "",
```

```
  edat: 0
```

```
}
```

```
const persona = persona_template;
```

```
persona.nom="Jhon";
```

```
persona.edat = 25;
```

```
const persona2 = persona_template;
```

```
persona2.nom="Silvia";
```

```
persona2.edat = 55;
```

```
console.log("edat", edat);
```

```
console.log("edatNou", edatNou);
```

```
console.log("Persona 1", persona);
```

```
console.log("Persona 2", persona2);
```

CALL STACK

Adress	Value
0001	35
0002	36
0003	D30F

HEAP

Adress	Value
D30F	{ nom: "Silvia", edat: 55 }

Les variables *persona_template*, *persona* i *persona2* apunten al mateix objecte que està al HEAP

```
edat 36
```

```
edatNou 35
```

```
Persona 1 ▶ Object { nom: "Sylvia", edat: 55 }
```

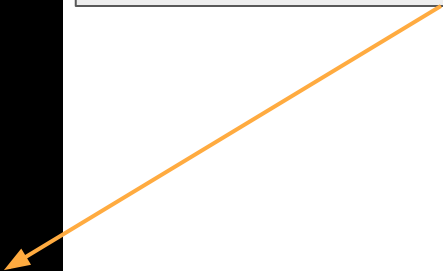
```
Persona 2 ▶ Object { nom: "Sylvia", edat: 55 }
```

Característica	Objectes	Classes
Reutilització	No permet fer instàncies	Sí
Herència	Difícil	Sí
Getters/Setters	No	Sí
Eficiència memòria	Mètodes duplicats	Optimitzat (prototype)

- Si el projecte és petit i no requereix instàncies múltiples, pots usar objectes literals.
- Si necessites reutilització, escalabilitat i herència, les classes són la millor opció.

```
class Persona {  
  // Li definim un constructor  
  constructor(nom, edat) {  
    // Atributs de la classe  
    this.nom = nom;  
    this.edat = edat;  
  }  
  
  // Mètode de la classe  
  presentar() {  
    console.log(`Hola, em dic ${this.nom} i tinc ${this.edat}  
anys.`);  
  }  
}  
  
//Cridem una instància amb el new  
const personal = new Persona("Anna", 30);  
console.log(personal);  
personal.presentar();
```

Una instància d'una classe és un objecte (com els que hem estat treballant), però amb els avantatges propis de la classe.



```
► Object { nom: "Anna", edat: 30 }  
Hola, em dic Anna i tinc 30 anys.
```



```
class Persona {  
  constructor(nom, edat) {  
    this.nom = nom;  
    this.edat = edat;  
  }  
  
  presentar() {  
    console.log(`Hola, em dic ${this.nom} i tinc ${this.edat} anys.`);  
  }  
}  
  
// Creem una primera instància de la classe  
const personal = new Persona("Anna", 30);  
personal.presentar();  
  
// Creem una segona instància de la classe  
const persona2 = new Persona("Carles", 50);  
persona2.presentar();
```

**Al permetre diferents instàncies
no repetim codi i tenim un codi
més net i modular.**

```
Hola, em dic Anna i tinc 30 anys.
```

```
Hola, em dic Carles i tinc 50 anys.
```

```
class Persona {
  #edat;
  constructor(nom, edat) {
    this.nom = nom;
    this.#edat = edat;
  }
  presentar() {
    console.log(`Hola, em dic ${this.nom} i tinc ${this.edat} anys.`);
  }
  get edat(){
    return this.#edat;
  }
  set edat(novaEdat){
    this.#edat = novaEdat;
  }
}

// Creem una primera instància de la classe
const personal = new Persona("Anna", 44);
personal.presentar();
personal.edat = 38;
personal.presentar();
```

Els getters i setters permeten controlar l'accés a les propietats d'un objecte.

Hola, em dic Anna i tinc 44 anys.

Hola, em dic Anna i tinc 38 anys.

[illegible]

```
class Client {  
    #nif; // Propietat privada  
  
    constructor(nom, nif) {  
        this.nom = nom;  
        this.#nif = nif;  
    }  
  
    mostrarInfo() {  
        console.log(`El nom de l'usuari és ${this.nom} amb NIF ${this.#getNifProtegit()}.`);  
    }  
  
    //Mètode privat  
    #getNifProtegit(){  
        return this.#nif = "*****" + this.#nif.slice(5);  
    }  
}  
  
const client1 = new Client("Joan", "12345678Z");  
client1.mostrarInfo();  
// client1.#nif; No s'hi pot accedir  
// client1.#getNifProtegit(); No s'hi pot accedir
```

**Propietats i mètodes privats
eviten modificacions accidentals
des de fora de la classe.**

El nom de l'usuari és Joan amb NIF *****678Z.

```
//Classe Filla
class Gos extends Animal{
  constructor(nom, tipus, raca){
    super(nom, tipus);
    this.raca = raca;
  }

  //Sobreescriure el mètode getSoroll
  getSoroll () {
    console.log("Els gossos lladren!");
  }

  getDescripcio () {
    console.log(`El nom del gos ${this.nom} i és de la raça ${this.raca}`);
  }
}

const gos1 = new Gos("Jackie", "Gos", "Border collie");
console.log(gos1);
console.log(gos1.getDescripcio());
```

```
// Classe Pare
class Animal {
  constructor(nom, tipus) {
    this.nom = nom;
    this.tipus = tipus;
  }

  // Mètode públic
  getSoroll () {
    console.log("Els animals fan sorolls...");
  }
}
```

L'herència en classes facilita la reutilització de funcionalitats i atributs.

Permet sobreescriure els mètodes.

```
► Object { nom: "Jackie", tipus: "Gos", raca: "Border collie" }
El nom del gos Jackie i és de la raça Border collie
```

```
class Matematica {  
    static sumar(a, b) {  
        return a + b;  
    }  
  
    static restar(a, b) {  
        return a - b;  
    }  
}  
  
// No fa falta fer una instància i sempre la pots utilitzar  
const operacio1 = Matematica.sumar(5, 10);  
const operacio2 = Matematica.restar(7, 5);  
console.log(operacio1);  
console.log(operacio2);
```

**Atributs i funcions que pots
utilitzar sense tenir una
instància de la classe.**

**Molt útil per agrupar
funcionalitats i atributs d'un
mateix concepte**

- Per agrupar dades + funcionalitat
- Per crear moltes instàncies amb la mateixa estructura
- Per reutilitzar codi amb herència
- Per separar **responsabilitats**
 1. Capa de Servei/API
 2. Capa Models
 3. Capa de Persistencia
 4. Capa de Presentació/Controlador (UI Logic)

▼ SPOTIFYBUSCADOR

> .vscode

▼ env

JS env.js

▼ script

▼ api

JS SpotifyAPI.js

▼ models

JS Album.js

JS Item.js

JS ItemsList.js

JS Podcast.js

JS Track.js

▼ utils

JS const.js

JS playlist.js

JS spotify.js

> style

◆ .gitignore

<> index.html

<> playlist.html

Definim la informació sensible com *client* i el *secret* de Spotify

Definim tota la funcionalitat relacionada amb l'APIRest

Definim les estructures de l'aplicació

Definim les constants de l'aplicació. (URLS dels endpoints,...)

Quan es crea qualsevol objecte en javascript, aquest objecte sempre hereta un objecte *prototype*. (Si mirem els objectes en javascript contenen una propietat *prototype*)

Prototype és un objecte que conté mètodes i propietats i son accessibles pels objectes que creem i utilitzem (ja que els hereta)

```
//CLASSE
class Person {
  constructor(nom, edat){
    this.nom = nom;
    this.edat = edat;
  }
  presentar(){
    console.log(`Hola, soc ${this.nom}`);
  }
}

//INSTÀNCIA DE LA CLASSE
const objClass = new Person("Carles", 37);
```

```
//OBJECTE
const objSimple = {
  raca: "Border collie",
  vacunat: true
}
```

```
//OBJECTE ARRAY
const objArray = [1,2,3]
```

```
//OBJECTE DATA
const objData = new Date();
```

```
console.log(objClass);
console.log(objSimple);
console.log(objArray);
console.log(objData);
```

```
▼ Object { nom: "Carles", edat: 37 }
  edat: 37
  nom: "Carles"
  <prototype>: Object { ... }

▼ Object { raca: "Border collie", vacunat: true }
  raca: "Border collie"
  vacunat: true
  <prototype>: Object { ... }

▼ Array(3) [ 1, 2, 3 ]
  0: 1
  1: 2
  2: 3
  length: 3
  <prototype>: Array []

▼ Date Sat Feb 08 2025 11:22:39 GMT+0100 (Hora estàndard d'Europa central)
  <prototype>: Date.prototype { ... }
```

```
//OBJECTE ARRAY  
const objArray = [1,2,3];  
console.log(objArray);
```

Quan faig servir un array (és un objecte) els mètodes que puc utilitzar pertanyen a l'objecte prototype que hereta

```
▼ Array(3) [ 1, 2, 3 ]  
  0: 1  
  1: 2  
  2: 3  
  length: 3  
  <prototype>: Array []  
    ▶ at: function at()  
    ▶ concat: function concat()  
    ▶ constructor: function Array()  
    ▶ copyWithin: function copyWithin()  
    ▶ entries: function entries()  
    ▶ every: function every()  
    ▶ fill: function fill()  
    ▶ filter: function filter()  
    ▶ find: function find()  
    ▶ findIndex: function findIndex()  
    ▶ findLast: function findLast()  
    ▶ findLastIndex: function findLastIndex()  
    ▶ flat: function flat()  
    ▶ flatMap: function flatMap()  
    ▶ forEach: function forEach()  
    ▶ includes: function includes()  
    ▶ indexOf: function indexOf()  
    ▶ join: function join()
```

```
  ▶ keys: function keys()  
  ▶ lastIndexOf: function lastIndexOf()  
    length: 0  
  ▶ map: function map()  
  ▶ pop: function pop()  
  ▶ push: function push()  
  ▶ reduce: function reduce()  
  ▶ reduceRight: function reduceRight()  
  ▶ reverse: function reverse()  
  ▶ shift: function shift()  
  ▶ slice: function slice()  
  ▶ some: function some()  
  ▶ sort: function sort()  
  ▶ splice: function splice()  
  ▶ toLocaleString: function toLocaleString()  
  ▶ toReversed: function toReversed()  
  ▶ toSorted: function toSorted()  
  ▶ toSpliced: function toSpliced()  
  ▶ toString: function toString()  
  ▶ unshift: function unshift()  
  ▶ values: function values()  
  ▶ with: function with()  
  ▶ Symbol(Symbol.iterator): function values()  
  ▶ Symbol(Symbol.unscopables): Object { at: true, copyWithin: true, entries: true, ... }  
  <prototype>: Object { ... }
```

Els mètodes que podrà utilitzar l'objecte creat pertanyen a l'objecte prototype i seran accessibles gràcies a l'herència

```
objArray.  
  concat (met  
  copyWithin  
  entries  
  every  
  fill  
  filter  
  find  
  findIndex  
  flat  
  flatMap  
  forEach  
  includes
```

En els casos de les classes, al crear una instancia de la classe, l'objecte creat hereta el prototype.

```
class Person {  
  constructor(nom, edat){  
    this.nom = nom;  
    this.edat = edat;  
  }  
  
  presentar() {  
    console.log(`Hola, soc  
    ${this.nom}`);  
  }  
}  
  
const persona1 = new Person("Carles", 37);  
console.log(persona1);
```

Els atributs es guarden a l'objecte

Els mètodes, getters i setters en el prototype

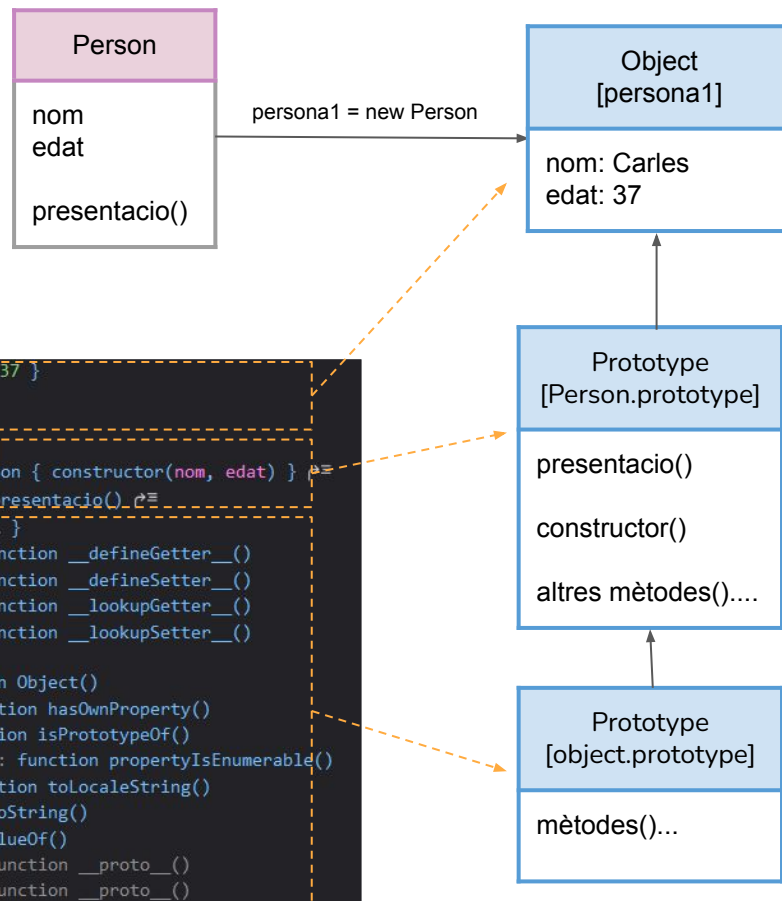
```
▼ Object { nom: "Carles", edat: 37 }  
  edat: 37  
  nom: "Carles"  
  <prototype>: Object { ... }  
    ► constructor: class Person { constructor(nom, edat) }  
    ► presentar: function presentar()  
    ► <prototype>: Object { ... }  
    Cadena de prototypes
```

El prototype que hereta l'objecte:

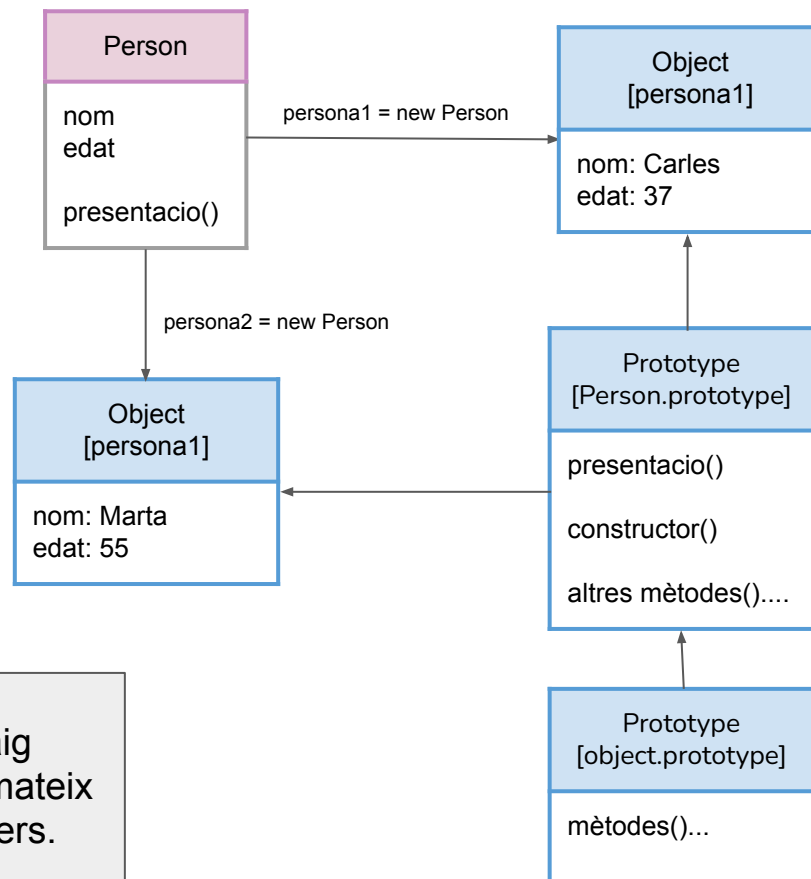
- Conté el mètode **constructor**
- **Guarda els mètodes i els getter i setters de la classe.**
- **Prototype chain:** L'objecte prototype pot heretar altres classe prototypes (cadena de prototypes) fins arribar a l'objecte prototype que sempre serà l'últim.

```
class Person {  
  constructor(nom, edat) {  
    this.nom = nom;  
    this.edat = edat;  
  }  
  
  presentacio() {  
    console.log(`Hola, soc ${this.nom}`);  
  }  
}  
  
const personal1 = new Person("Carles", 37);  
  
console.log(personal1);
```

Quan creem una instància, els atributs es guarden en l'objecte de la *Person*. En canvi, els mètodes i els getters/setters es guarden en l'objecte *prototype* que hereta.



```
class Person {  
  constructor(nom, edat) {  
    this.nom = nom;  
    this.edat = edat;  
  }  
  
  presentacio() {  
    console.log(`Hola, soc ${this.nom}`);  
  }  
}  
  
const persona1 = new Person("Carles", 37);  
const persona2 = new Person("Marta", 55);
```



Un dels avantatges del prototype és que quan faig diferents instàncies d'una mateixa classe hereta el mateix prototype on hi tenim els mètodes i els gettes/setters.

Els *arrays* i els *sets* son estructures de dades que permet emmagatzemar col·leccions de dades.

Cada un d'ells té unes característiques i casos d'ús diferents.

Array

```
const array = ["a", "b", "c"];  
console.log(array);
```

```
▼ Array(3) [ "a", "b", "c" ]  
  0: "a"  
  1: "b"  
  2: "c"  
  length: 3  
  ► <prototype>: Array []
```

- Permet valors duplicats
- Permet valors ordenats
- Accés per índex
- Menys eficients que el set.
- Gran quantitat de mètodes per manipular dades

Útil utilitzar-ho quan és necessari manipular dades (ordenar, filtrar,...) i que pugui tenir valors duplicats.

Set

```
const setObj = new Set(["a", "b", "c"]);  
console.log(setObj);
```

```
▼ Set(3) [ "a", "b", "c" ]  
  size: 3  
  ► <entries>  
  ► <prototype>: Set.prototype { ... }
```

- No permet duplicats (si n'hi ha els elimina de forma interna)
- No permet valors ordenats
- Molt eficient

Útil quan és necessari un gran rendiment i no és necessari manipular dades ni tenir valors duplicats.

Mètodes que retornen un nou array i no modifiquen l'array actual

Mètode	Desc	Codi	array	Sortida
map()	Aplica una funció a cada element	<code>const newArray = array.map((x) => x * 2);</code>	[1, 2, 3, 4]	[2, 4, 6, 8]
filter()	Filtra elements que compleixen una condició	<code>const newArray = array.filter((x) => x > 2);</code>	[1, 2, 3, 4]	[3, 4]
slice()	Retorna una part de l'array	<code>const newArray = array.slice(1);</code>	[1, 2, 3, 4]	[2, 3, 4]
		<code>const newArray = array.slice(1, 3);</code>	[1, 2, 3, 4]	[2, 3]
		<code>const newArray = array.slice(-2);</code>	[1, 2, 3, 4]	[3, 4]
concat()	Concatena arrays o valors	<code>const newArray = array.concat([5, 6]);</code>	[1, 2, 3, 4]	[1, 2, 3, 4, 5, 6]
flat()	Aplana un array multidimensional	<code>const newArray = [1, [2, 3], [4]].flat();</code>	[1, [2, 3], [4]]	[1, 2, 3, 4]
flatMap()	Combina map() i flat()	<code>const newArray = array.flatMap((x) => x * 2);</code>	[1, [2, 3], [4]]	[2, 4, 6, 8]

Mètodes que **modifiquen** l'array actual

Mètode	Desc	Codi	array	Sortida
forEach()	Executa una funció per a cada element	<i>arr.forEach((valor, index, array) => {array[index] = valor + 1;});</i>	[1, 2, 3, 4]	[2, 3, 4, 5]
push()	Afegeix elements al final de l'array	<i>array.push(5);</i>	[1, 2, 3, 4]	[1, 2, 3, 4, 5]
unshift()	Afegeix elements al principi de l'array	<i>array.unshift(0);</i>	[1, 2, 3, 4]	[0, 1, 2, 3, 4]
pop()	Elimina l'últim element de l'array	<i>array.pop();</i>	[1, 2, 3, 4]	[1, 2, 3]
shift()	Elimina el primer element de l'array	<i>array.shift();</i>	[1, 2, 3, 4]	[2, 3, 4]
splice()	Afegeix o elimina elements en qualsevol posició. (definim l'inici i el número d'elements a eliminar)	<i>array.splice(2, 1);</i>	[1, 2, 3, 4]	[1, 2, 4]
reverse()	Inverteix l'ordre dels elements de l'array	<i>array.reverse();</i>	[1, 2, 3, 4]	[4, 3, 2, 1]
fill()	Omple l'array amb un valor específic	<i>array.fill(1);</i>	[1, 2, 3, 4]	[1, 1, 1, 1]

Mètodes que **modifiquen** l'array actual

Mètode	Desc	Codi	array	Sortida
sort()	Ordena els elements de l'array (asc) de forma numèrics	<code>array.sort((a, b) => a - b);</code>	[2, 1, 4, 3]	[1, 2, 3, 4]
	Ordena els elements de l'array (desc) de forma numèrics	<code>array.sort((a, b) => b - a);</code>	[2, 1, 4, 3]	[4, 3, 2, 1]
	Ordena els elements de l'array (asc) de forma alfabètica	<code>array.sort((a,b) => { if (a > b) return 1; if (a < b) return -1; });</code>	["b", "a", "d", "c"]	["a", "b", "c", "d"]
	Ordena els elements de l'array (desc) de forma alfabètica	<code>array.sort((a,b) => { if (a > b) return -1; if (a < b) return 1; });</code>	["b", "a", "d", "c"]	["d", "c", "b", "a"]

Mètodes de **cerca** a l'array actual

Mètode	Desc	Codi	array	Sortida
find()	Retorna el primer element que compleix una condició.	<code>array.find(n => n > 2);</code>	[1, 2, 3, 4]	3
findIndex()	Retorna la posició del primer element que compleix una condició	<code>array.findIndex(n => n > 2);</code>	[1, 2, 3, 4]	2
indexOf()	Retorna la primera posició on es troba un element. Retorna -1 si no existeix	<code>array.indexOf(4);</code>	[1, 2, 3, 4]	3
some()	Retorna true si almenys un element compleix la condició	<code>array.some(n => n > 3);</code>	[1, 2, 3, 4]	true
every()	Retorna true si tots els elements compleixen la condició.	<code>array.every(n => n > 0);</code>	[1, 2, 3, 4]	true
includes()	Retorna true si l'element existeix, false si no.	<code>array.includes(3);</code>	[1, 2, 3, 4]	true

Javascript permet extreure la informació dels arrays d'una forma més ràpida i flexible.

```
//Array de valors  
const arr = [1,2,3];  
  
const a = arr[0];  
const b = arr[1];  
const c = arr[2];  
console.log(a,b,c);
```

1 2 3

Amb destructuring ho podem fer amb una sola línia.

```
//Array de valors  
const arr = [1,2,3];  
const [a,b,c] = arr;  
console.log(a,b,c);
```

1 2 3

```
const arr = [1,2,3];  
  
const a = arr[0];  
const b = arr[2];  
console.log(a,b);
```

1 3

En aquest cas només volem el primer i el tercer valor.

```
//Array de valors  
const arr = [1,2,3];  
const [a, ,b] = arr;  
console.log(a,b);
```

1 3

```
const arr = [1,2,[3,4]];  
const [a, , [b,c]] = arr;  
console.log(a,b,c);
```

1 3 4

Podem tenir destructuring dins d'un altre destructuring

Javascript permet extreure la informació dels objectes d'una forma més ràpida i flexible.

```
const usuari = {  
  nom: "john",  
  any_naixement: 1999,  
  habilitats: ["JavaScript", "HTML", "CSS"],  
  contacte: {  
    email: "john@example.com",  
    telefon: "123456789"  
  }  
}
```

Amb destructuring ho podem
fer amb una sola línia.

```
//Destructuring de l'objecte principal  
//Ha de tenir el mateix nom que la propietat  
const {nom, any_naixement, habilitats, contacte} = usuari;  
console.log("destructuring de l'obj principal");  
console.log(nom, any_naixement, habilitats, contacte);  
  
//Destructuring de l'objecte contacte  
const {email, telefon} = contacte;  
console.log("destructuring de l'obj contacte");  
console.log(email, telefon);  
  
//Destructuring de l'array habilitats  
const [hab1, hab2, hab3] = habilitats;  
console.log("destructuring de l'array habilitats");  
console.log(hab1, hab2, hab3);
```

```
destructuring de l'obj principal  
john 1999 ▶ Array(3) [ "JavaScript", "HTML", "CSS" ] ▶ Object { email: "john@example.com", telefon: "123456789" }  
destructuring de l'obj contacte  
john@example.com 123456789  
destructuring de l'array habilitats  
JavaScript HTML CSS
```